

RV-MalTrace 当前进展

35T 硬件追踪证据链与论文推进路线

主题： RISC-V 硬件辅助行为追踪

类型： 论文进展汇报

核心问题： 当前能做到什么、边界在哪里、下一步做什么

2026 年 5 月 24 日

汇报提纲

- 一 **研究定位**——把 trace tap 放进论文故事线
- 二 **已完成能力**——仿真、35T 板级证据与语义恢复
- 三 **证据边界**——现在能说什么，哪些还不能说
- 四 **下一阶段**——从原型闭环推进到论文贡献

本次报告的重点不是罗列文件——而是把当前仓库整理成可审阅的科研进展、风险边界和下一步 gate。

Outline

- ① 研究定位
- ② 已完成能力
- ③ 证据边界
- ④ 下一阶段

这不是一个单纯的 RTL demo

当前仓库已经从“在 RISC-V/CVA6 或 LiteX/VexRiscv 上加一个 trace tap”推进到可复现实验证据链：事件格式、仿真 golden、35T 板级运行、行为恢复、解释接口和 claim boundary 都有对应文件与检查脚本。

对论文进展报告而言，最稳妥的表达是：RV-MalTrace 现在可以支撑一个 35T / LiteX / VexRiscv 硬件追踪辅助的恶意行为证据链原型。这不是成熟检测器，也不是 CVA6 板级结论，但已经足够作为下一步论文路线的工程底座。

一句话进展：当前原型已完成受控行为矩阵的板级 trace 证据闭环，并开始把真实恶意代码参考中的行为抽象迁移到安全可执行样例。

当前技术链路（图 1）

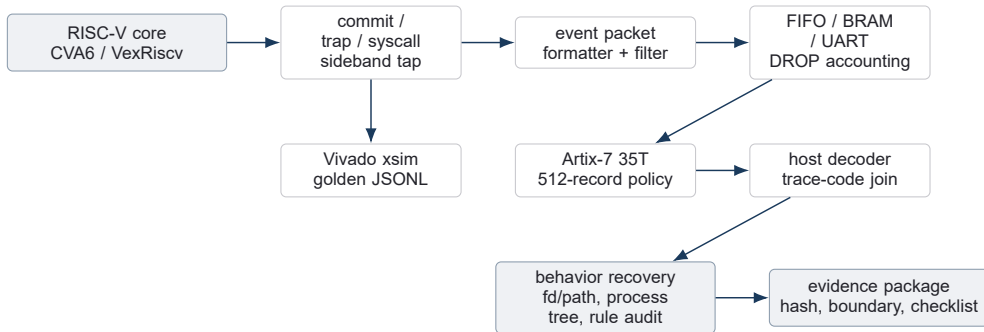


图 1 当前仓库覆盖从硬件事件采集到行为证据包的完整原型链路。

这条链路的关键点是低扰动事件采集和离线证据解释分离：硬件侧只保留提交事件与必要上下文，复杂语义在 host 侧恢复并受 gate 约束。

Outline

- ① 研究定位
- ② 已完成能力**
- ③ 证据边界
- ④ 下一阶段

仓库已经形成的四层能力（表 1）

层次	现在已经能做的事	主要证据
trace RTL / 格式	记录已提交、控制流、syscall、trap、context、ARG_MEM、DROP 与 MARKER，并有过滤、压缩原型、DROP 计数。	trace_format.md
Vivado / CVA6 仿真	trace unit、RVFI adapter、direct-core CVA6、full SoC probe、RV64GC microprobe 均有 PASS 记录。	sim_results.md
35T 板级实验	Artix-7 35T / LiteX / VexRiscv 下，13 个受控样例 full matrix 通过 512-record gate。	35T evidence bundle
行为解释与论文证据	支持 code map、trace-code join、fd/path、process tree、real-malware-derived lineage 和 claim boundary。	paper evidence chain

这四层已经让项目从“能跑一个样例”变成“能解释、能复现、能限制说法”的研究原型。

Trace 事件模型已经稳定

当前事件模型围绕已提交行为设计，而不是完整 load/store 记录。syscall entry 从 U-mode ECALL 的 trap path 捕获，syscall return 从 SRET 返回 U-mode 捕获，DROP 作为一等事件进入 trace，避免在带宽不足时把 core 停住。

事件族	代表事件	用于回答的问题
控制流	RETIRE, BRANCH, JUMP	执行路径、压缩指令长度、跳转目标
syscall	SYSCALL_ENTRY, SYSCALL_RET	参数、返回值、持续周期、系统调用序列
异常与上下文	TRAP, CSR, SATP, PRIV	trap cause、地址空间、特权级切换
资源边界	ARG_MEM, DROP, MARKER	指针快照、溢出可见性、样例作用域

设计取舍：默认不追踪完整内存访问；需要语义时用 syscall-scoped ARG_MEM 或 side-channel 进行有界补全。

Vivado 与 CVA6 仿真门禁 (表 2)

门禁	状态	说明
trace unit + RVFI adapter	PASS	覆盖 syscall return、pointer string、pointer guardrails、backpressure、filter、board minimal。
CVA6 direct-core matrix	PASS	smoke、branch、jump、ecall、illegal trap、ebreak 等 trace-on/no-trace 终态一致。
full SoC harness probes	PASS	breakpoint smoke、UART/MMIO store path、normal tohost/MMIO completion 已记录。
RV64GC microprobe	PASS	I/M/A/F/D/C 每类至少一条 committed instruction probe 通过。
Linux / arbitrary program	未声称	当前不是 Linux 全系统任意程序正确性结论，也不是 CVA6 物理板结论。

这部分适合在报告中说明：CVA6 仿真方向已有可复现基础，但论文级板上 Linux 语义仍要单独 gate。

35T 主矩阵已经闭环（表 3）

指标	结果	解释
run id	35t-smallcap-r512 full matrix 20260521	主 evidence bundle，用于严格 trace gate。
样例矩阵	13/13 PASS	benign + synthetic malware-like，全矩阵 gate 通过。
trace 容量	512 records	并非通过扩大到 1024 records 解决；当前按 small-capacity policy 分样例配置。
trace health	UNKNOWN = 0	marker scope、runtime process attribution、DROP/capacity 均进入 gate；corrupt event 也为 0。
语义补全	selected PASS	targeted validation 关闭 fd/path 与 process-tree 代表性解释路径。

这里最有价值的是 “35T 小容量约束下仍可建立证据链”，不是成熟恶意软件检测能力。

真实恶意代码派生行为有六行证据（表 4）

行为行	来源	状态	边界
ELF header probe	DarthRa reference	PASS	真实恶意代码派生行为，不是 payload 等价执行。
rootkit device probe	DarthRa reference	PASS	安全控制下的行为验证。
virus fixture walk	DarthRa reference	PASS	文件遍历类行为抽象。
proc scan	Mirai reference	PASS	非网络版本，排除外联。
watchdog probe	Mirai reference	PASS	非网络行为抽象。
encoded table	Mirai reference	PASS	字符串/表行为抽象。

可讲法：这些结果支持“选取自真实恶意代码参考的行为可以被 35T 原型 trace、验证和规则审计”，不支持“真实家族检测准确率”。

解释接口让结果可复述

现在的结果不是只能看原始 UART 或 JSONL。仓库 目前解释层已经覆盖三类代表性语义：已经提供终端解释接口，把 trace health、gate 状态、恢复出的行为、可疑 cue、证据来源和 non-claim 一起打印。

命令	用途
rvmt explain:35t	单样例解释
--flow	run-level 过程视图
--format markdown	生成可提交材料

1. **code map**——PC 与 ELF symbol / syscall site 关联；
2. **fd/path**——代表性 openat -> fd -> read/write/close 闭环；
3. **process tree**——clone 返回 child PID 与 wait PID 匹配。

Outline

- ① 研究定位
- ② 已完成能力
- ③ 证据边界**
- ④ 下一阶段

现在可以稳妥陈述的结论（表 5）

可陈述结论	精确含义	证据位置
35T 原型闭环	Artix-7 35T / LiteX / VexRiscv 上，受控样例的硬件 trace、运行时归因、规则审计已闭环。	application closure
主矩阵 gate 通过	512-record policy 下，13 个样例 13/13 PASS，UNKNOWN/corrupt 为 0。	full matrix bundle
代表性语义闭环	fd/path 与 process-tree 代表路径已在 targeted validation 中 PASS。	semantic summaries
真实恶意派生行为可追踪 证据链纪律可用	DarthRa/Mirai reference 的 6 行安全控制行为验证 PASS。 hash manifest、claim boundary、review checklist 已形成 CCF-A-style discipline。	lineage / baseline strong chain package

这些说法的共同特点是：限定平台、限定样例、限定安全控制、限定证据类型。

仍然不能越界的说法（表 6）

不能声称	状态	原因
CVA6 物理板验证完成	NO	当前 35T 结果来自 LiteX/VexRiscv, CVA6 主要是仿真和综合/资源路径。
真实恶意软件检测准确率	NO	真实恶意派生行为不是家族分类、IOC/TTP 覆盖或检测质量评估。
完整语义恢复	NO	fd/path 与 process-tree 是代表性闭环; source-line、全局进程所有权、完整内存语义仍缺。
成熟检测器或产品级系统	NO	当前是 evidence-chain prototype, 规则审计是应用展示, 不是成熟 detector。
单 trace side-channel 全门 禁 PASS	NO	targeted validation 是 dual-channel; side-channel 不是 strict full-matrix trace gate。

建议表述：先讲已闭环证据，再主动讲 non-claims。这样能清楚呈现项目是“有边界的进展”，不是夸大。

资源与非干扰证据的边界（表 7）

指标	Baseline	Trace-enabled	Delta
LUT	84,923	125,717	+40,794 (+48.04%)
FF	56,491	59,301	+2,810 (+4.97%)
BRAM18 equiv	108	108	+0
DSP	27	27	+0
Slack	0.177 ns	0.177 ns	0.000 ns

非干扰 gate 已检查 sideband-only、registered snapshot、DROP-not-stall、direct-core trace/no-trace final PASS 等条件。资源结果可以引用为当前 routed report 的 trace-enabled delta。

边界：这些数据不能被包装成 IPC/Fmax 提升或真实 workload overhead 结论；它们说明的是当前实现成本和非反压原则。

Outline

- ① 研究定位
- ② 已完成能力
- ③ 证据边界
- ④ 下一阶段

论文主线要从 prototype 走向 contribution

如果目标只是“硬件能采 syscall event”，研究贡献会偏工程。更有论文竞争力的主线应当是：RISC-V 低扰动硬件辅助 syscall 语义追踪，其中硬件 trace 提供 committed、低可见度的行为事实，离线语义层恢复 fd/path/process/memory-permission 等行为图。

下一阶段的中心问题不是再堆更多样例，而是回答：仅靠寄存器与上下文能恢复到哪里；哪些 pointer 参数必须补全；硬件 pointer snapshot、kernel helper/eBPF fallback 与 baseline 方法的边界分别是什么。

论文故事线：从“35T 原型可行”推进到“RISC-V hardware-assisted semantic behavior tracing with bounded pointer reconstruction and evasion-aware evaluation”。

下一步一：把 pointer 语义做成核心创新

许多关键 syscall 的行为语义不在寄存器数值本身，而在 pointer 指向的用户内存中。当前 ARG_MEM 已有 synthetic openat pathname 与 guardrails 回归，下一步要把它推进到 Linux/CVA6 信号接入或明确 fallback。

$$\text{SYSCALL_ENTRY}(a_7, a_0, \dots, a_5) + \text{ARG_MEM}(ptr, bytes) \\ \Rightarrow \text{openat}(\text{path}), \text{execve}(\text{argv}), \text{connect}(\text{sockaddr})$$

路径	要验证的点	风险
hardware snapshot	S-mode copy-from-user load 能否稳定捕获字符串/结构体片段。	LSU 信号、跨页、带宽
kernel helper/eBPF	只补 pid/path/string，不替代硬件 trace。	威胁模型收窄
offline reconstruction	fd/path/process/memory behavior graph 形成可审计输出。	完整性边界

下一步二：把对比实验补齐（表 8）

baseline	比较目标	关键指标
strace/ptrace	最直接的软件 syscall ground truth 与 anti-debug 可见性。	overhead, detectability
eBPF-only / helper	OS helper 路线的语义能力与扰动。	semantic accuracy, overhead
QEMU / plugin	模拟器路径的可控性与真实执行差距。	timing, coverage
software instrumentation	编译期或二进制插桩与硬件旁路采集的对比。	perturbation, completeness
RV-MalTrace ablation	event-only、pointer snapshot、helper 三种模式拆开比较。	accuracy, resource, drop

对系统安全论文而言，实验不能只证明“能 trace”，还要证明为什么硬件辅助比常规软件方法有价值。

未来两周优先级（表 9）

任务	交付物	判断标准
source-line attribution	在 code map 中保留 DWARF/source line，并 join 到代表 syscall/trap site。	source summary 从 PARTIAL 推进
Linux semantic gate	选择 2–3 个 open/exec/mmap/process workload，固定 runbook 与 ground truth。	path/process 语义可复查
pointer snapshot 决策	CVA6 LSU 信号 map + synthetic-to-Linux feasibility note。	hardware/fallback 路线明确
baseline plan	strace, QEMU, eBPF/helper 的可运行脚本和指标表。	同一样例可横向比较
paper claim table	将 allowed claims / forbidden claims 变成论文式 contribution boundary。	可直接审阅

优先级原则：先补“论文里一定会被问”的证据缺口，再扩样例规模。

需要讨论的三个决策

接下来需要尽早确定方向，否则后续实验容易发散：

1. **论文定位**——主打 malware analysis，还是更稳地表述为 RISC-V semantic behavior tracing，并把 malware 作为应用场景；
2. **语义补全路线**——优先冲 hardware pointer snapshot，还是 hardware trace + trusted helper 双路径并行；
3. **目标强度**——当前 35T evidence chain 是否作为阶段性小论文/技术报告，还是继续补 CVA6/Linux/对比实验后冲更高层级会议。

建议路线：短期保守声称 35T evidence-chain prototype；中期把 pointer semantic reconstruction 与 evasion-aware baseline 做成论文核心贡献。

